RNN小史 — 記憶を手渡す機械

参考資料 — 注意の前に、世界を回していた記憶の機械

rkskmt

この資料について

- 本編は MLP（固定窓）から一気に注意へ進んだ。歴史はその間に **RNN の時代**（1990s〜2017年ごろ）を挟んでいる
- RNNの強みと、なぜ主役を降りたかを、地名データの実測で追体験する資料
- 前提は「窓を広げる」（MLP言語モデル）と「小さなGPTを訓練する」（早期終了）まで

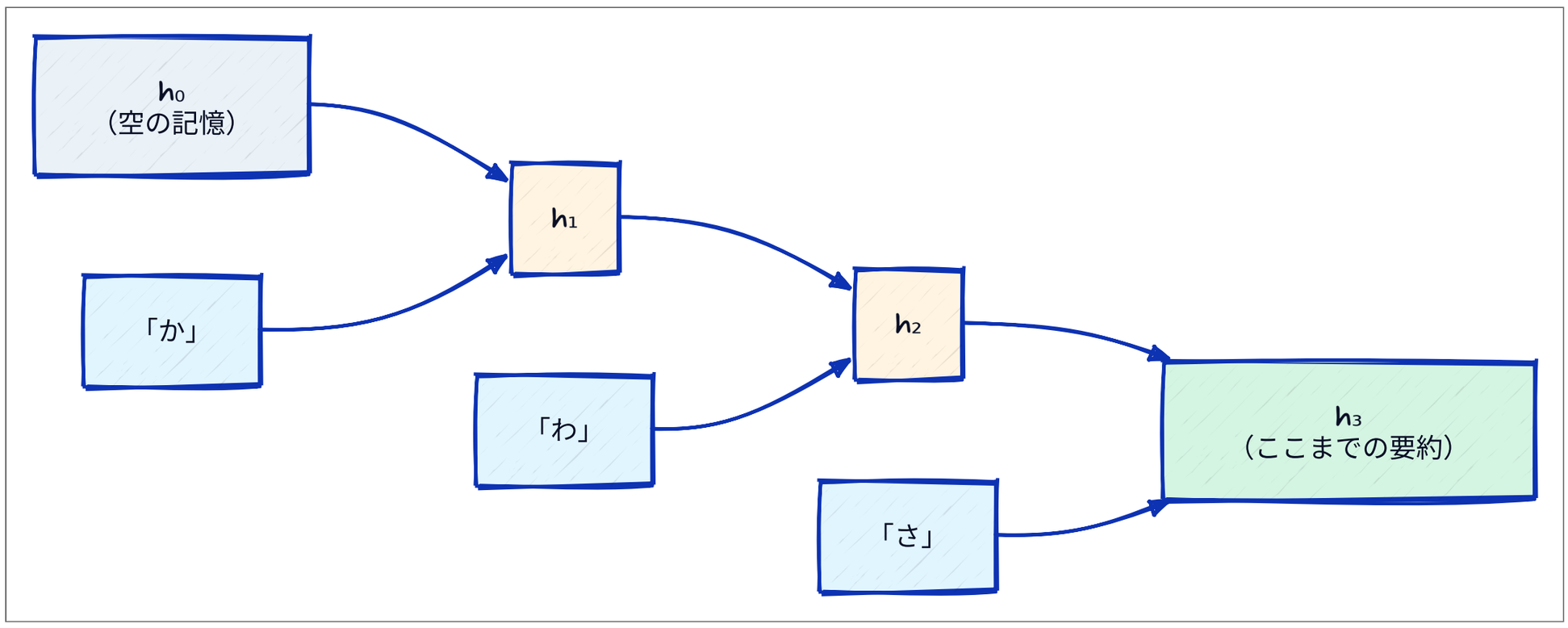
窓の外は見えない — MLP の宿題

- MLP言語モデルは直前 ctx 文字の**固定窓**だった — 窓より前は、存在しないのと同じ
- 「窓を広げる」の回の実測：窓を広げても val はわずかしか良くならず、**過学習が先に来た**
- そこで出てくる欲：**文の最初から全部を、長さに関係なく持ち歩けないか？**

アイデア：記憶を1本のベクトルにする

- 文字を1つ読むたびに、**記憶ベクトル h** を更新して次の位置へ手渡す：

$$h_t = \tanh(W_x\, x_t + W_h\, h_{t-1})$$



- 読み終わったときの h が「ここまで全部の要約」 — **窓が要らない**
- この「自分の出力を自分に戻す」構造をまとめて **RNN**（**Recurrent Neural Network**；再帰型）と呼ぶ

部品はぜんぶ知っている

- 新しい数学はゼロ：埋め込み・行列・tanh（シグモイドの親戚のS字カーブ）・softmax
- 違いは**ループが構造の中にある**ことだけ — forward の中に「1文字ずつ」の for 文が入る

Code

```
import pandas as pd
import torch
import torch.nn as nn
import torch.nn.functional as F

torch.set_num_threads(1)

class ElmanRNN(nn.Module):
    def __init__(self, V, n_embd=8, hidden=64):
        super().__init__()
        self.emb = nn.Embedding(V, n_embd)
        self.wx = nn.Linear(n_embd, hidden)
        self.wh = nn.Linear(hidden, hidden)
        self.out = nn.Linear(hidden, V)
        self.hidden = hidden

    def forward(self, x):                          # x: (B, T)
        e = self.emb(x)
        h = torch.zeros(x.shape[0], self.hidden)
        logits = []
        for t in range(x.shape[1]):                # 1文字ずつ h を更新
            h = torch.tanh(self.wx(e[:, t]) + self.wh(h))
            logits.append(self.out(h))
        return torch.stack(logits, dim=1)          # (B, T, V)
```

- これは最古参の型（**Elman ネット**、1990年）。部品数は約1万個 — 地名MLPと同じ階級

実験：地名メーカーを RNN で

chimei.csv を **data/** に置き、いつもの地名ストリームで訓練する（500歩・約30秒）。

Code

```
names = pd.read_csv("data/chimei.csv")["yomi"].tolist()
chars = ["・"] + sorted(set("".join(names)))
stoi = {c: i for i, c in enumerate(chars)}
ids = []
for name in names:
    ids += [0] + [stoi[c] for c in name]
ids.append(0)
stream = torch.tensor(ids)

torch.manual_seed(0)
model = ElmanRNN(len(chars))
opt = torch.optim.Adam(model.parameters(), lr=3e-3)
g = torch.Generator().manual_seed(0)
for step in range(1, 501):
    ix = torch.randint(len(stream) - 17, (256,), generator=g)
    x = torch.stack([stream[i:i + 16] for i in ix])
    y = torch.stack([stream[i + 1:i + 17] for i in ix])
    opt.zero_grad()
    loss = F.cross_entropy(model(x).view(-1, len(chars)), y.view(-1))
    loss.backward()
    opt.step()
print(f"500歩後の損失: {loss.item():.3f}")
```

Result

```
500歩後の損失: 2.077
```

聞いてみる — 架空地名、RNN 版

- 生成も1文字ずつ：h を空にして始め、引いた文字を入れてはまた引く

Code

```
g = torch.Generator().manual_seed(1)
for _ in range(10):
    h = torch.zeros(1, 64)
    i, name = 0, ""
    while True:
        h = torch.tanh(model.wx(model.emb(torch.tensor([i]))) + model.wh(h))
        i = torch.multinomial(F.softmax(model.out(h), dim=-1)[0], 1, generator=g).item()
        if i == 0:
            break
        name += chars[i]
    print(name)
```

Result

```
おおのつちょう
みつしべつし
ぬまじし
あわじしらちょう
みなみやまきし
うらがはし
たいねきむら
とよさつし
あいきし
つかいししべし
```

- 「おおのつちょう」「ぬまじし」…… 10件とも実在しない新作 — **耳で聞くかぎり MLP に負けていない**

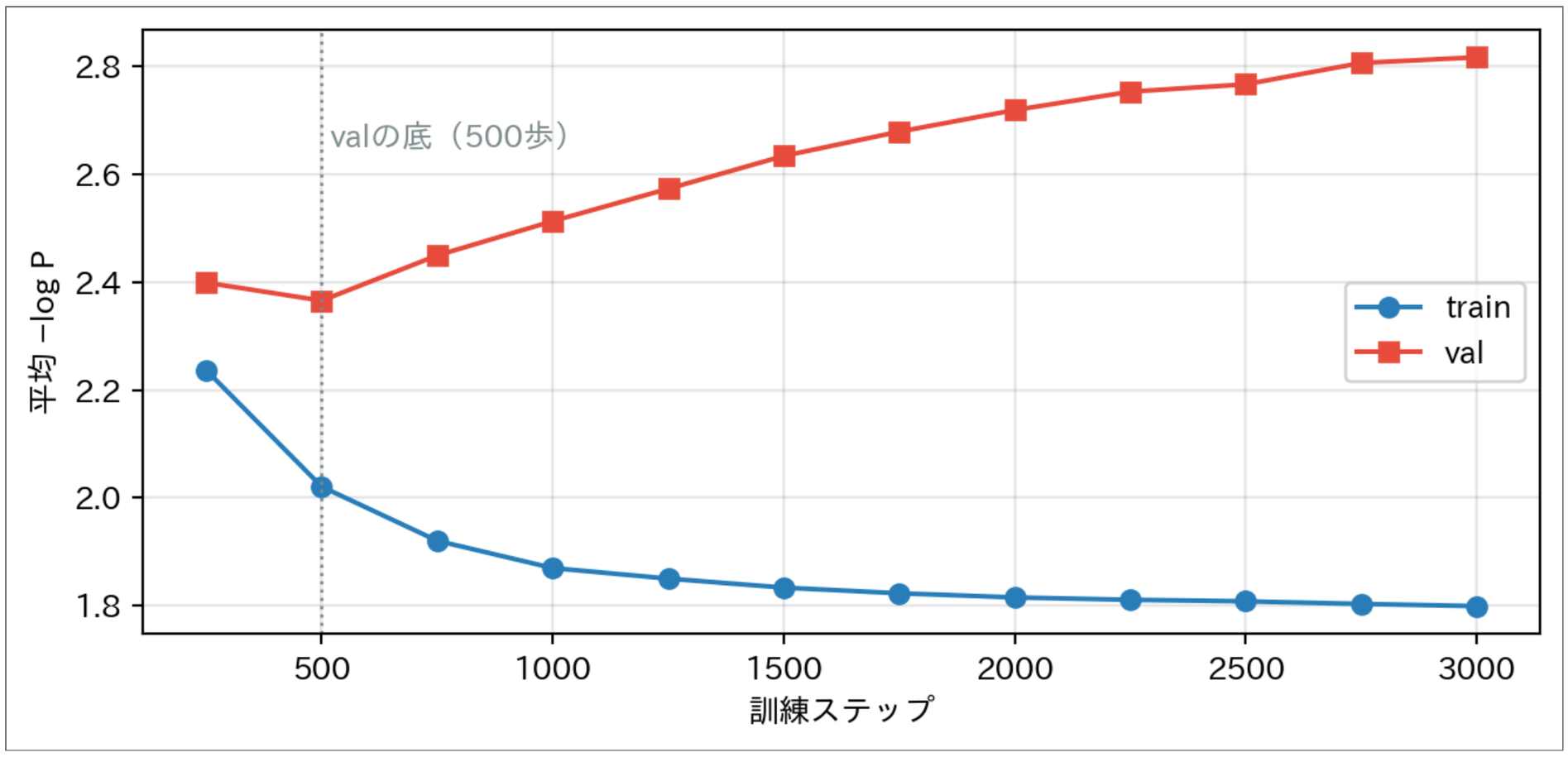
成績表：RNN、勝つ

同じ分け方（9:1）・同じ採点（1遷移あたりの平均 $-\log P$）での勝負：

モデル	記憶の形	val
数えるだけ（バイグラム）	直前1文字	2.62
MLP（窓2）	直前2文字	2.59
RNN（valの底）	**ここまで全部**	==**2.37**==

- 窓なしの記憶は本当に効く — **RNNは実力で主役だった**
- では、なぜ君たちの GPT に RNN は入っていないのか。理由は3つ、すべて「訓練のしにくさ」

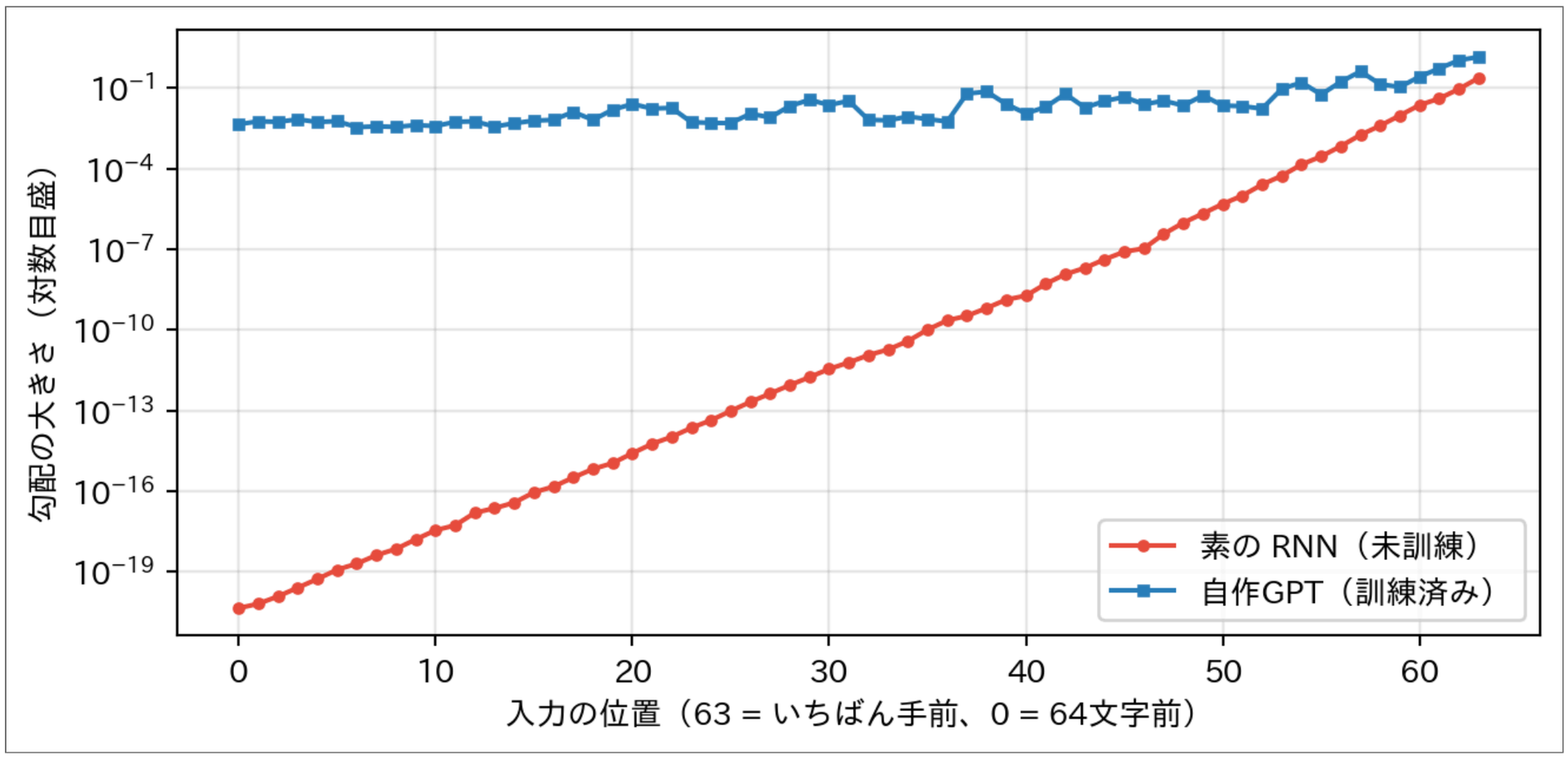
つまずき① 過学習が速い



- 底は **500歩・2.37**。回し続けると一直線に悪化し、3000歩では **2.82**（train は 1.80）— バイグラムにも負ける
- 対策は知ってのとおり**早期終了** — これは道具で解決できた。次の2つが本物の壁

つまずき② バケツリレーで勾配が消える

- 記憶は毎歩、$\tanh$ と W_h の**同じ関所**を通る。誤差が入口へ**戻る**ときも同じ関所を逆に通る
- 64文字さかのぼるなら関所64回 — 1回ごとに少し痩せる勾配は、**指数関数的に消えていく**



- 最後の1文字の損失だけを流したとき、**64文字前に届く勾配は最後の位置の 10^{-20} 倍**（素の RNN、実測）— **学び始める前から、遠くが学べない構造**
- 自作GPT は 64文字前でも 10^{-3} 程度 — **視線（注意）は距離に橋を架ける**

LSTM — 水門で記憶を守る

- **LSTM（Long Short-Term Memory、1997）**：記憶ベクトルに**ゲート**（何を通し、何を忘れるかの水門）を付けた改良型
- 消える勾配がかなり和らぎ、RNN は実用の主役になった — 2010年代の音声認識・機械翻訳（seq2seq）は LSTM の天下
- それでも、最後のつまずきだけは構造上どうにもならなかった

つまずき③ 1文字ずつしか進めない

- h_t は h_{t-1} 待ち — **今日の1歩は、昨日の1歩が終わるまで始まらない**
- 並列計算機（GPU）は「同時にたくさん」が取り柄なのに、RNN の訓練はその取り柄を使えない

Result

```
forward 1回（32文×64文字、1スレッドCPU):
  RNN（1層のループ64歩）    : 1692 ms
  自作GPT（2ブロック+FFN）: 437 ms
```

- 幅をそろえた RNN の**1層ぶんのループ**が、自作GPT**全身**の3〜4倍遅い（このCPU実装・1スレッドの実測）。GPU では差が桁になる
- 注意は全位置を**行列積ひと振り**で処理する — この並列性が、データと計算を積む**物量作戦**への切符だった

交代劇：2017年

- 注意はもともと、LSTM 翻訳機の**補助輪**として生まれた（どの単語を見るかだけを担当）
- 2017年の **Attention Is All You Need** が「補助輪だけで走れる。RNN は要らない」と示す — それが君たちの組んだ **Transformer**
- RNN は引退したわけではない：音声・省電力デバイスでは現役。そして「並列に訓練できる新しい再帰」（Mamba など）の研究も続いている — 物語はまだ終わっていない

まとめ

- RNN は**実力で主役だった** — 地名では窓付き MLP に勝つ（val 2.37 vs 2.59）
- 降板の理由は精度ではなく**訓練のしにくさ**：消える勾配（10^{-20} のバケツリレー）と、並列化できない逐次性
- 注意はこの2つを一挙に解決した — 遠くへ直接橋を架け、全位置を一括計算する
- だから本編は「窓 → 注意」の最短路を歩いた。歴史は回り道の中で、同じ2つの壁を確かめていた